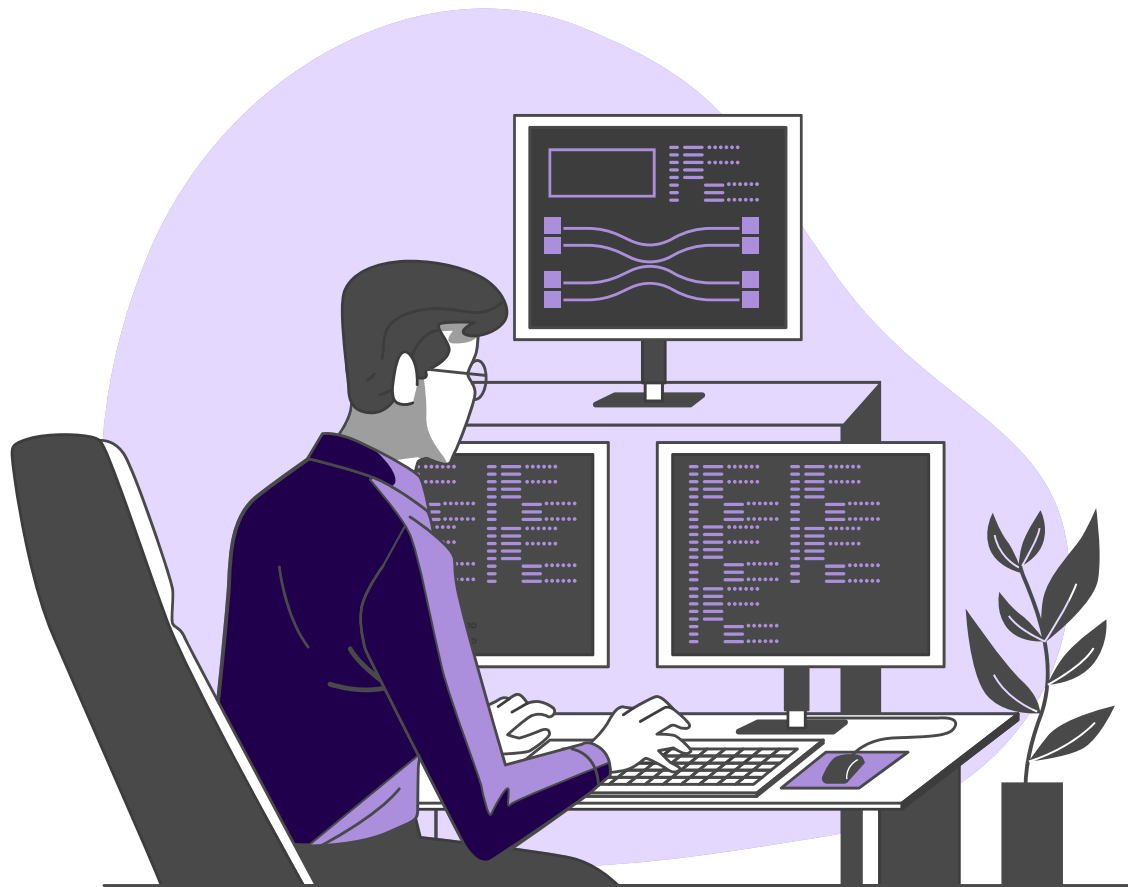
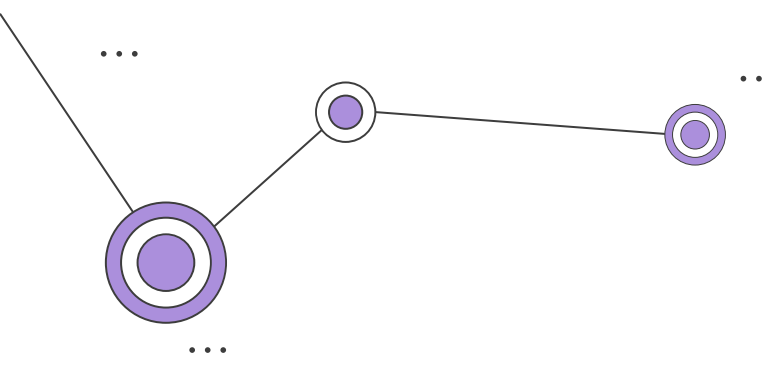




SQL for Data Analysis

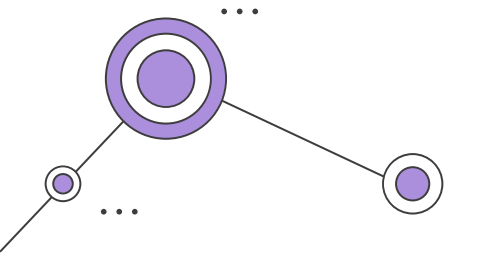
Practical Techniques for
Cleaning, Joining, and
Analyzing Data



Kimberly Fessel, PhD

Data Educator

Founder of Dr Kim Data



Learning Objectives



Clean and transform data directly in SQL



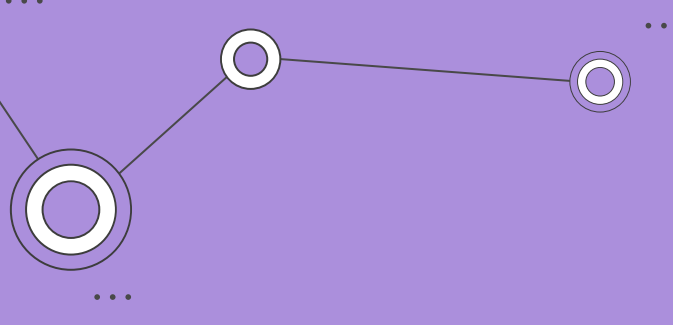
Combine and query from multiple tables



Perform complex **aggregations and rankings**

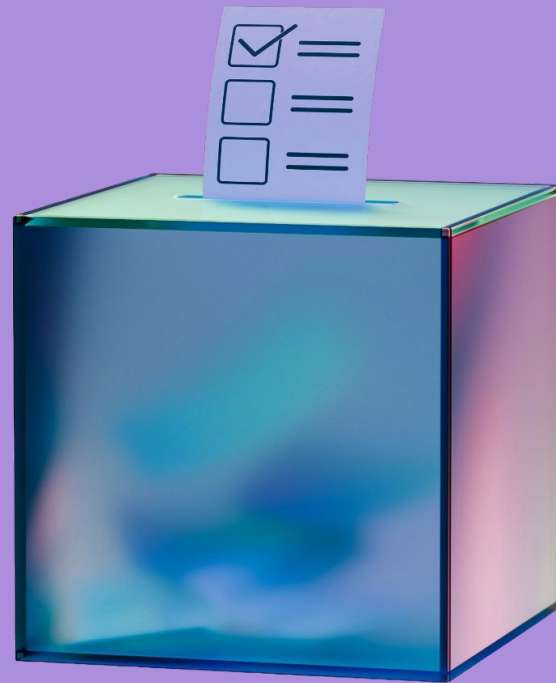
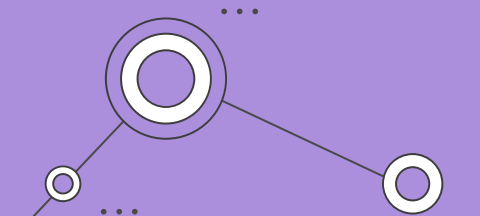


Solve **real-world data problems** through common analysis patterns



Poll #1

What most often trips you
up in SQL?



Course Overview

01

Data Cleaning

Missing values • Casting
Strings and dates • CASE

02

Finding & Joining

Subqueries • CTEs
Joins • Unions

03

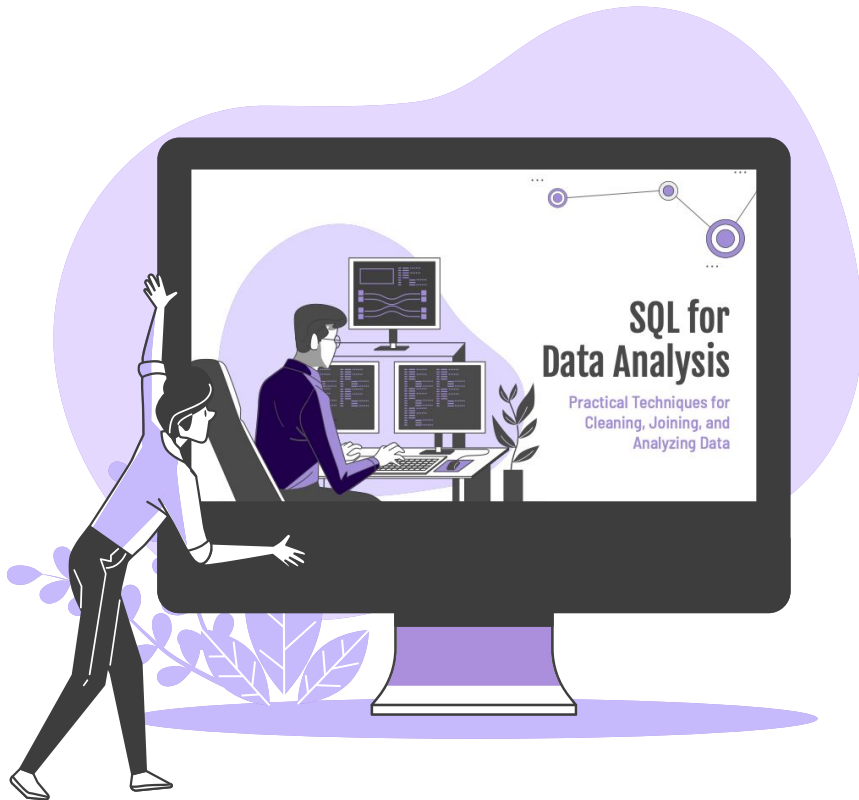
Window Functions

Basics • Ranking
Summary stats • Running totals

04

Analysis Patterns

Top-N • Outlier detection
Cohort retention • MoM growth



Course Materials

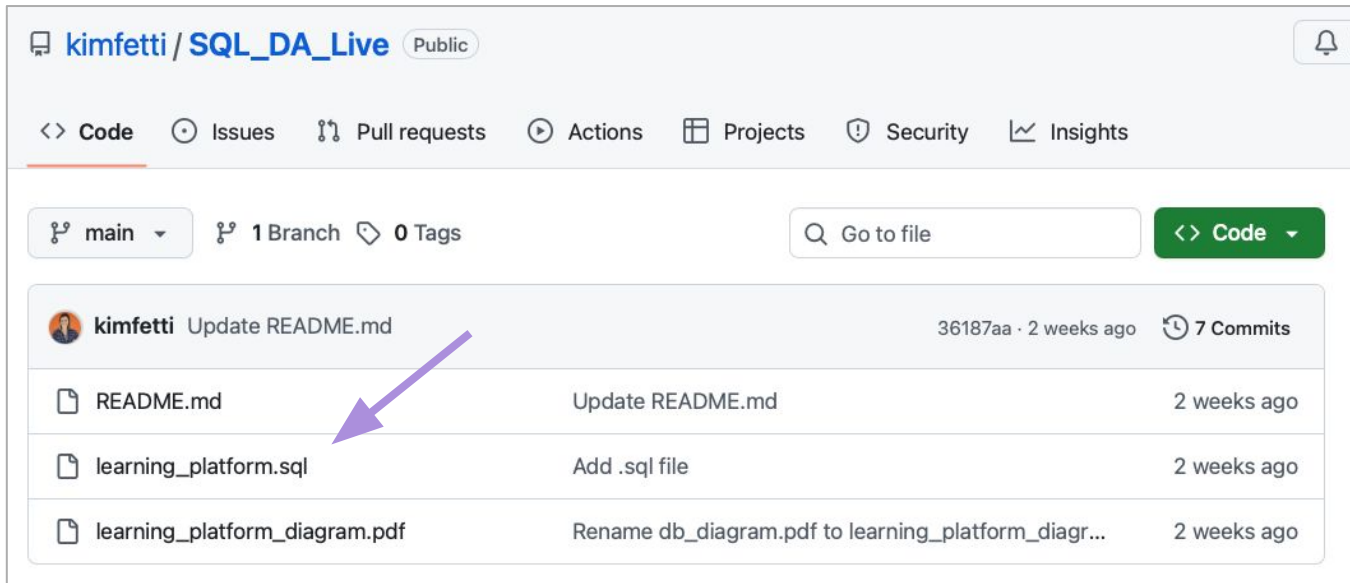


Course materials available on GitHub

github.com/kimfetti/SQL_DA_Live

Course Materials

github.com/kimfetti/SQL_DA_Live



kimfetti / **SQL_DA_Live** Public

<> Code Issues Pull requests Actions Projects Security Insights

main 1 Branch 0 Tags

Go to file

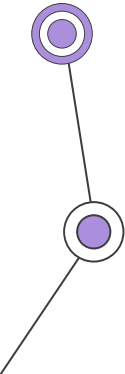
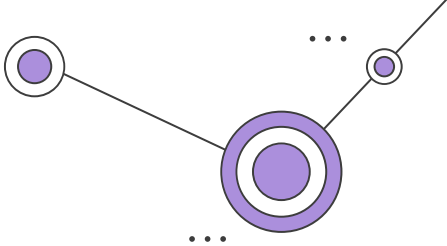
<> Code

kimfetti Update README.md 36187aa · 2 weeks ago 7 Commits

README.md	Update README.md	2 weeks ago
learning_platform.sql	Add .sql file	2 weeks ago
learning_platform_diagram.pdf	Rename db_diagram.pdf to learning_platform_diagr...	2 weeks ago

Course Materials

github.com/kimfetti/SQL_DA_Live



SQL_DA_Live / learning_platform.sql

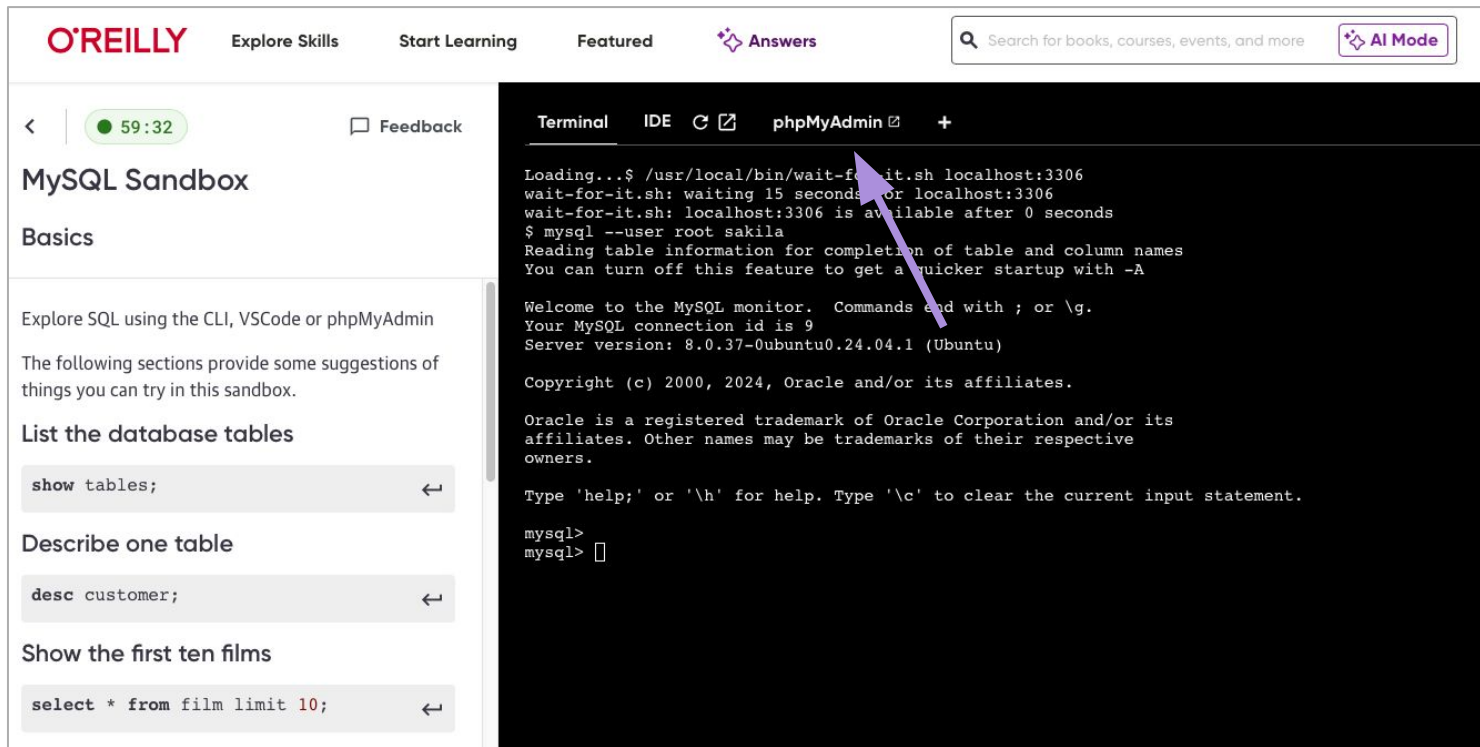
 kimfetti Add .sql file 07a4e35 · 2 weeks ago History

Code Blame Raw Copy Download Edit View Source

```
1  -- Drop database if it exists (so the script is re-runnable)
2  DROP DATABASE IF EXISTS learning_platform;
3  CREATE DATABASE learning_platform;
4  USE learning_platform;
5
6  -- USERS
7
8  -- Drop table if it exists (safe if script re-run)
9  DROP TABLE IF EXISTS users;
10
11 -- Create users table
12 CREATE TABLE users (
```


Course MySQL Sandbox

Course Tools



O'REILLY Explore Skills Start Learning Featured Answers

Search for books, courses, events, and more AI Mode

< 59:32 Feedback

MySQL Sandbox

Basics

Explore SQL using the CLI, VSCode or phpMyAdmin

The following sections provide some suggestions of things you can try in this sandbox.

List the database tables

```
show tables;
```

Describe one table

```
desc customer;
```

Show the first ten films

```
select * from film limit 10;
```

Terminal IDE phpMyAdmin

```
Loading...$ /usr/local/bin/wait-for-it.sh localhost:3306
wait-for-it.sh: waiting 15 seconds for localhost:3306
wait-for-it.sh: localhost:3306 is available after 0 seconds
$ mysql --user root sakila
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.37-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

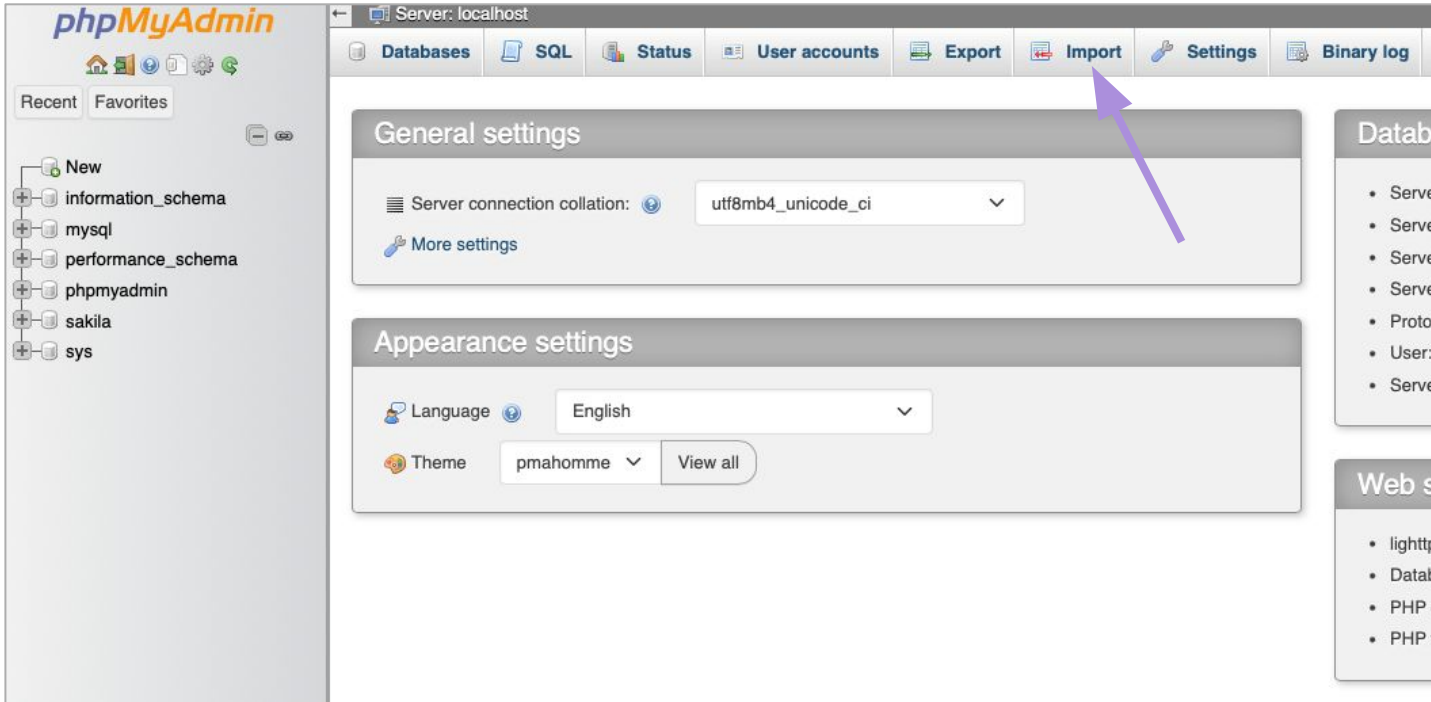
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
mysql>
```

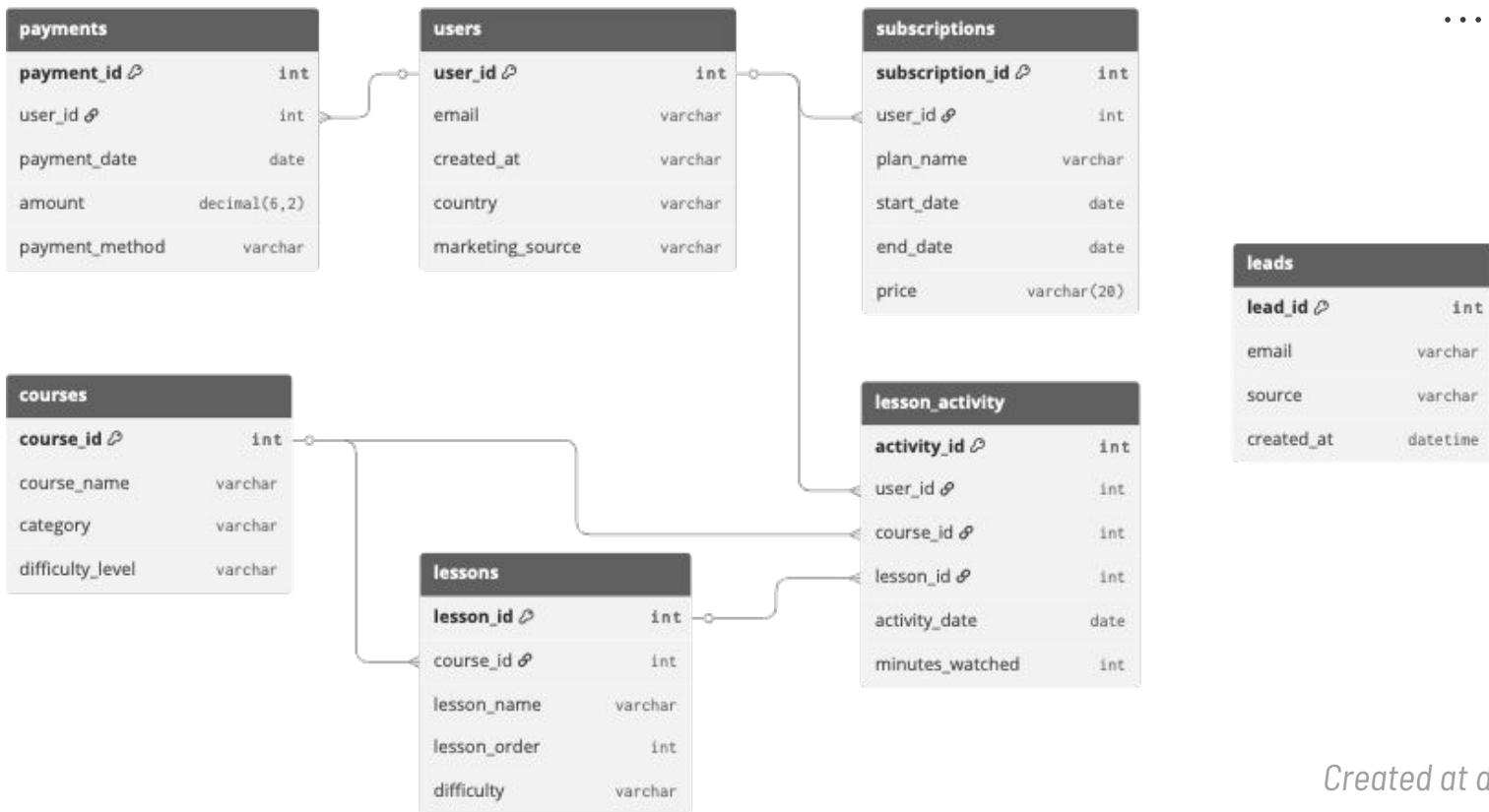
Course MySQL Sandbox

Course Tools



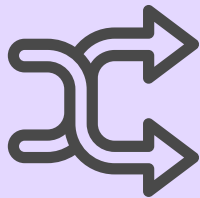
learning_platform

Database Diagram



01

Data Cleaning



Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice	?	2.00
Little Oven	cheese	
Chicago Chicago	meat lovers	4.50

Missing values may appear as:

- Blanks (NULL)

Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice	?	2.00
Little Oven	cheese	
Chicago Chicago	meat lovers	4.50

Missing values may appear as:

- Blanks (NULL)

Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice	?	2.00
Little Oven	cheese	
Chicago Chicago	meat lovers	4.50

Missing values may appear as:

- Blanks (NULL)
- Filler values
 - ?
 - N/A
 - -1

Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice	?	2.00
Little Oven	cheese	
Chicago Chicago	meat lovers	4.50

Missing values may appear as:

- Blanks (NULL)

IS NULL

- Filler values

- ?
- N/A
- -1

NULLIF()

Handling Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice		2.00
Little Oven	cheese	
Chicago Chicago	meat lovers	4.50

Multiple ways to handle:

- Leave missing data

Handling Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
Chicago Chicago	meat lovers	4.50

Multiple ways to handle:

- Leave missing data
- Drop missing data

Handling Missing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice	cheese	2.00
Little Oven	cheese	3.10
Chicago Chicago	meat lovers	4.50

Multiple ways to handle:

- Leave missing data
- Drop missing data
- Impute missing data

Determining **WHY** data is missing helps develop an appropriate strategy

Converting between Data Types

CAST allows you to convert one data type into another.

CAST keyword **AS keyword**

```
SELECT CAST (13.75 AS INT) ;
```

Expression **New data type**

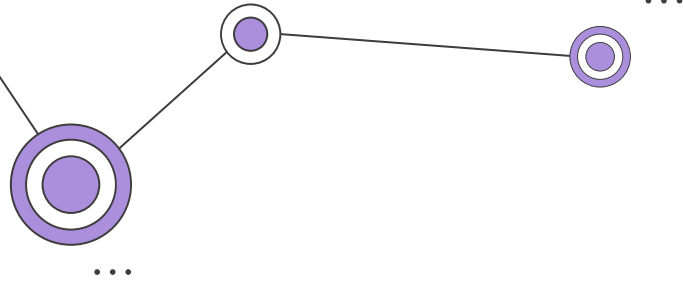
The diagram shows the SQL statement `SELECT CAST (13.75 AS INT) ;` on a light gray background. Above the word `CAST`, the text **CAST keyword** is written in purple with a purple arrow pointing to `CAST`. Above the word `AS`, the text **AS keyword** is written in purple with a purple arrow pointing to `AS`. Below the opening parenthesis and the number `13.75`, a purple curly bracket spans this area, with the text **Expression** written in purple below it. Similarly, below the words `AS INT`, a purple curly bracket spans this area, with the text **New data type** written in purple below it.

Converting between Data Types

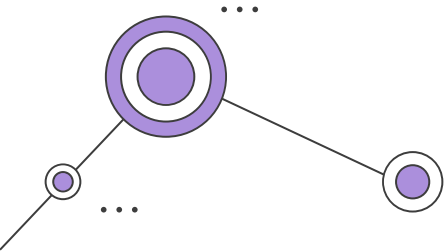
CAST allows you to convert one data type into another.

```
SELECT CAST(13.75 AS INT) ;
```

Result: 13



Coding Demonstration



Manipulating Strings

instructors

name

Dr. Kim.

DR KIM

dr. Kim

Top three data cleaning issues:

1. Remove “junk” characters
2. Trim white space from edges
3. Standardize case

Strings: REPLACE

1. Remove “junk” characters

REPLACE function

What to find

```
REPLACE (name, ' . ', ' ')
```

Column name

What to
replace it with

instructors

name

Dr. Kim.

DR KIM

dr. Kim

Strings: REPLACE

1. Remove “junk” characters

REPLACE function

What to find

```
REPLACE (name, ' . ', ' ')
```

Column name

What to
replace it with

instructors

name

Dr Kim

DR KIM

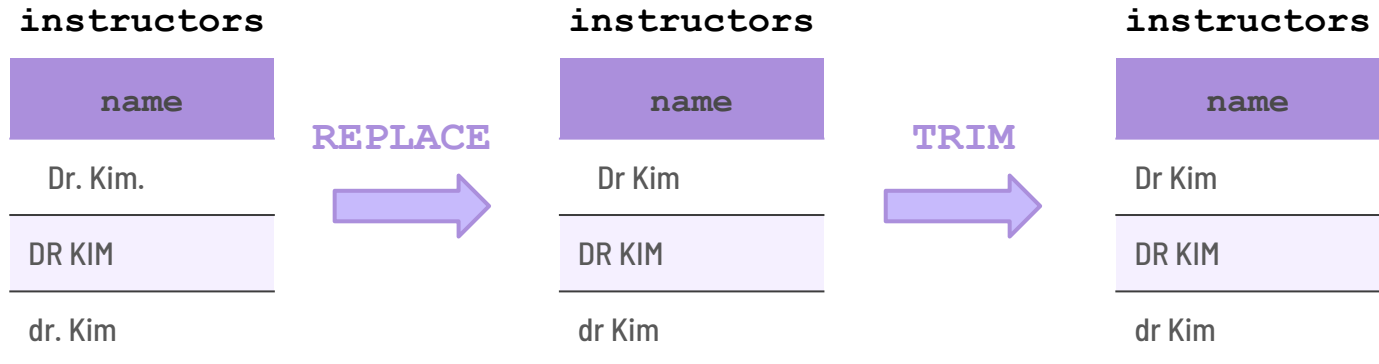
dr Kim

Strings: TRIM

2. Trim white space from edges

TRIM function

TRIM (REPLACE (name, ' . ', ' '))



Strings: TRIM

3. Standardize case

LOWER function

LOWER (TRIM (REPLACE (name, ' . ', ' ')))

instructors

name
Dr. Kim.
DR KIM
dr. Kim



instructors

name
dr kim
dr kim
dr kim

Manipulating Strings

Top three data cleaning issues:

1. Remove “junk” characters **REPLACE**
2. Trim white space from edges **TRIM**
3. Standardize case **LOWER / UPPER**

Working with Dates

Four common date techniques:

1. Convert strings to the `DATE` data type **CAST**
2. Extract date parts
3. Do date arithmetic
4. Convert dates to string formats

Date syntax varies
among RDBMSs.

Dates: EXTRACT

2. Extract date parts

EXTRACT function

FROM keyword

```
EXTRACT (YEAR FROM date_column)
```

Unit of time

Column or value

Dates: Arithmetic

3. Do date arithmetic

`date_column + INTERVAL 7 DAY`

INTERVAL keyword

Column or value

Math symbol

Amount of time

Dates: Arithmetic

4. Convert dates to string formats

DATE_FORMAT function

DATE_FORMAT (**date_column** , ' %m/%d/%Y ')

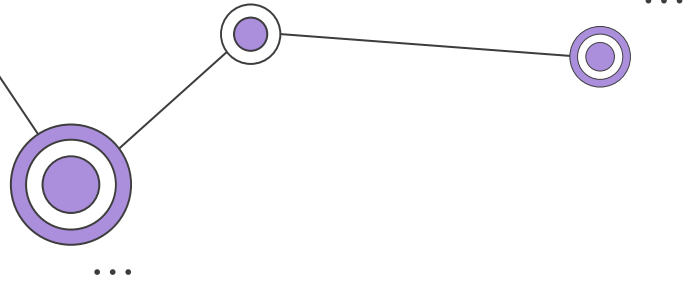
Column or value

Date format string

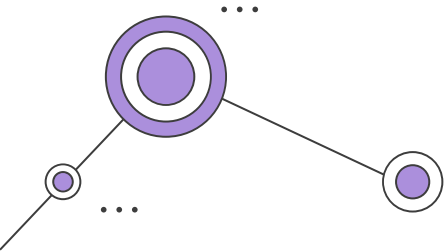
Working with Dates

Four common date techniques:

1. Convert strings to the DATE data type **CAST**
2. Extract date parts **EXTRACT**
3. Do date arithmetic **INTERVAL**
4. Convert dates to string formats **DATE_FORMAT**



Coding Demonstration



Bucketing Data

pizza

location	specialty	price
Sal's Pizza	pepperoni	3.50
Pie in the Sky	cheese	2.50
Nova Napoli	margherita	3.00
The Slice	cheese	2.00
Little Oven	cheese	3.10
Chicago Chicago	meat lovers	4.50

Bucketing Data

CASE Expression

pizza

location	specialty	price	price_range
Sal's Pizza	pepperoni	3.50	\$\$
Pie in the Sky	cheese	2.50	\$
Nova Napoli	margherita	3.00	\$\$
The Slice	cheese	2.00	\$
Little Oven	cheese	3.10	\$\$
Chicago Chicago	meat lovers	4.50	\$\$\$

CASE Expression

```
SELECT
CASE
  WHEN price < 3.00 THEN '$'
  WHEN price < 4.00 THEN '$$'
  ELSE '$$$'
END AS price_range
FROM pizza;
```

CASE keyword → CASE

Condition: price < 3.00 Result: '\$'

Continue to next if first is not met → WHEN price < 4.00 THEN '\$\$'

Final result for all others → ELSE '\$\$\$'

END keyword → END AS price_range

Alias → price_range

FROM pizza;

CASE Expression

Required keywords:

1. CASE
2. WHEN, THEN
3. END

SELECT

CASE

WHEN price < 3.00 THEN '\$'

WHEN price < 4.00 THEN '\$\$'

ELSE '\$\$\$'

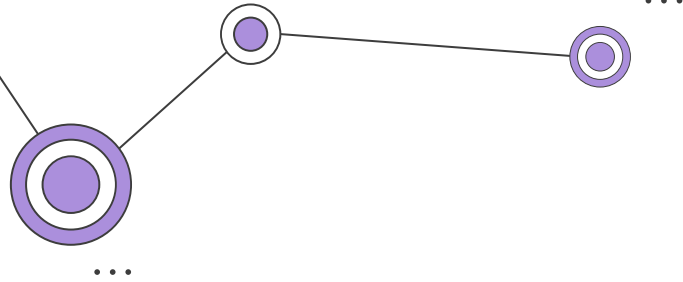
END AS price_range

FROM pizza;

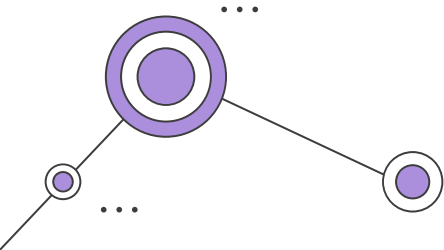
Bucketing Data

pizza

location	specialty	price	price_range
Sal's Pizza	pepperoni	3.50	\$\$
Pie in the Sky	cheese	2.50	\$
Nova Napoli	margherita	3.00	\$\$
The Slice	cheese	2.00	\$
Little Oven	cheese	3.10	\$\$
Chicago Chicago	meat lovers	4.50	\$\$\$



Coding Demonstration



5-Minute Break



02

Finding & Joining



Querying a Query

pizza

location	price
Sal's Pizza	3.50
Pie in the Sky	2.50
Nova Napoli	3.00
The Slice	2.00
Chicago Chicago	4.50

Which places sell a pizza slice for **less than average**?

```
SELECT * FROM pizza
```

```
WHERE
```

```
price < ??? ;
```

You need a query to calculate the average before completing the statement.

Querying a Query

pizza

location	price
Sal's Pizza	3.50
Pie in the Sky	2.50
Nova Napoli	3.00
The Slice	2.00
Chicago Chicago	4.50

What is the **average price** of a slice of pizza?

```
SELECT AVG(price)
```

```
FROM pizza;
```

Querying a Query

pizza

location	price
Sal's Pizza	3.50
Pie in the Sky	2.50
Nova Napoli	3.00
The Slice	2.00
Chicago Chicago	4.50

Which places sell a pizza slice for **less than average**?

```
SELECT * FROM pizza
```

```
WHERE
```

```
price < ??? ;
```



Querying a Query

pizza

location	price
Sal's Pizza	3.50
Pie in the Sky	2.50
Nova Napoli	3.00
The Slice	2.00
Chicago Chicago	4.50

Which places sell a pizza slice for **less than average**?

```
SELECT * FROM pizza
```

```
WHERE
```

```
price < (
```

```
SELECT AVG(price)
```

```
FROM pizza;
```

```
);
```

Querying a Query

pizza

location	price
Pie in the Sky	2.50
Nova Napoli	3.00
The Slice	2.00

Which places sell a pizza slice for **less than average**?

```
SELECT * FROM pizza
```

```
WHERE
```

```
price < (
```

```
    SELECT AVG(price)
```

```
    FROM pizza;
```

```
);
```

Subquery

A **subquery** is a query contained within another query.

```
SELECT ... WHERE ... (SELECT ...) ;
```

```
SELECT ..., (SELECT ...) FROM ...;
```

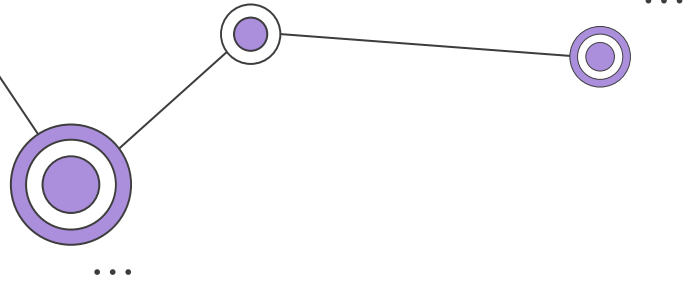
```
SELECT ... JOIN (SELECT ...) ...;
```


Derived Table

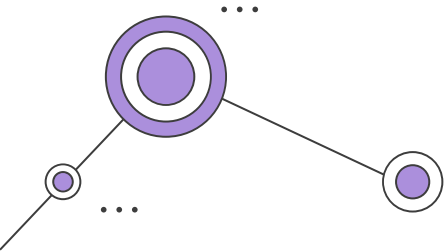
A **derived table** is a subquery that returns tabular information, often found within the FROM statement.

```
SELECT ... FROM (SELECT ...) ...;
```

Tabular:
returns multiple rows
and columns



Coding Demonstration



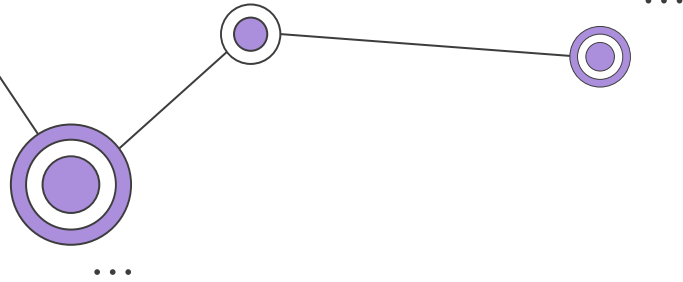
Common Table Expression (CTE)

A **common table expression (CTE)** provides an alternative to the derived table and uses **WITH**.

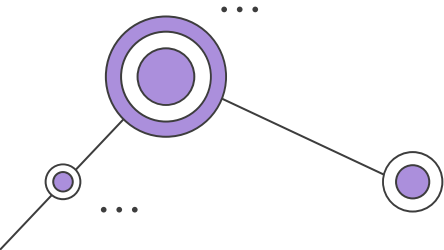
The diagram illustrates the syntax of a Common Table Expression (CTE) using two SQL statements. The first statement, `WITH my_cte AS (SELECT ...)`, is annotated with three labels: **WITH keyword** pointing to the `WITH` keyword, **CTE Name** pointing to `my_cte`, and **AS keyword** pointing to the `AS` keyword. The **CTE Definition** label is positioned above the `(SELECT ...)` clause. The second statement, `SELECT ... FROM my_cte;`, is annotated with the label **Query that references the CTE name** pointing to the `my_cte` in the `FROM` clause.

```
WITH my_cte AS (SELECT ...)
```

```
SELECT ... FROM my_cte;
```



Coding Demonstration



Derived Tables vs. CTEs

```
WITH o AS
  (SELECT * FROM orders
   WHERE price < 10.00)

SELECT o.prod_id, p.desc
FROM o JOIN products p
      ON o.prod_id = p.id;
```

- Reference CTE **multiple times** throughout your query

Derived Tables vs. CTEs

WITH o AS

```
(SELECT * FROM orders  
WHERE price < 10.00)
```

```
SELECT o.prod_id, p.desc  
FROM o JOIN products p  
ON o.prod_id = p.id;
```

- Reference CTE **multiple times** throughout your query

Derived Tables vs. CTEs

```
WITH o AS
  (SELECT * FROM orders
   WHERE price < 10.00)

SELECT o.prod_id, p.desc
FROM o JOIN products p
      ON o.prod_id = p.id;
```

- Reference CTE **multiple times** throughout your query
- Easier to read CTE queries

Derived Tables vs. CTEs

```
WITH my_cte1 AS (SELECT ...),  
     my_cte2 AS (SELECT ...)  
  
SELECT ...  
  
FROM my_cte1 JOIN my_cte2 ...;
```

- Reference CTE **multiple times** throughout your query
- Easier to read CTE queries, especially multiple CTEs

Derived Tables vs. CTEs

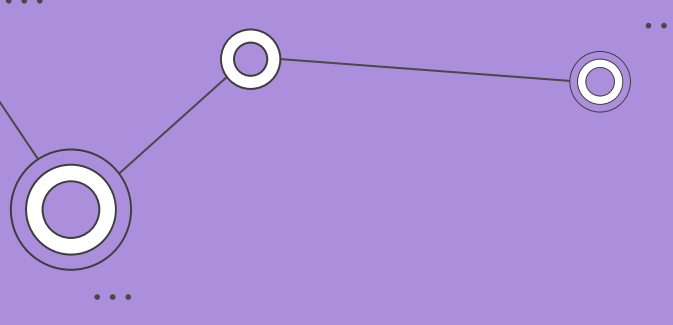
```
WITH my_cte1 AS (SELECT ...),  
     my_cte2 AS (SELECT ...)  
  
SELECT ...  
  
FROM my_cte1 JOIN my_cte2 ...;
```

- Reference CTE **multiple times** throughout your query
- Easier to read CTE queries, especially multiple CTEs

Derived Tables vs. CTEs

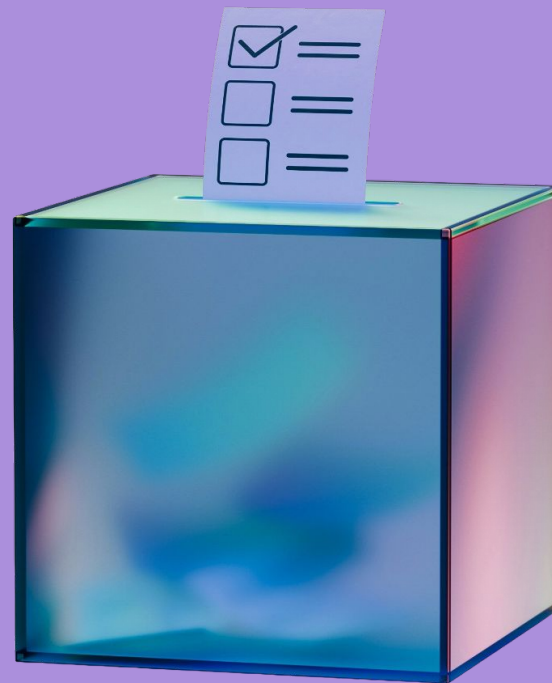
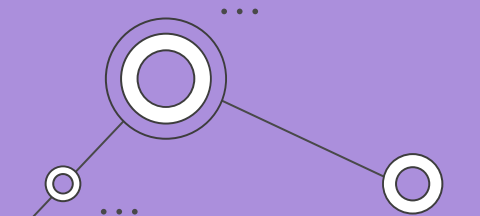
```
WITH my_cte1 AS (SELECT ...),  
     my_cte2 AS (SELECT ...)  
  
SELECT ...  
  
FROM my_cte1 JOIN my_cte2 ...;
```

- Reference CTE **multiple times** throughout your query
- Easier to read CTE queries, especially multiple CTEs
- CTEs may not work on very old SQL software



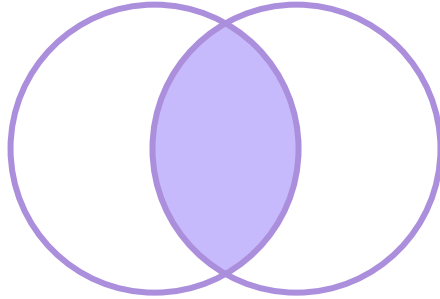
Poll #2

What do you find most
challenging about SQL joins?

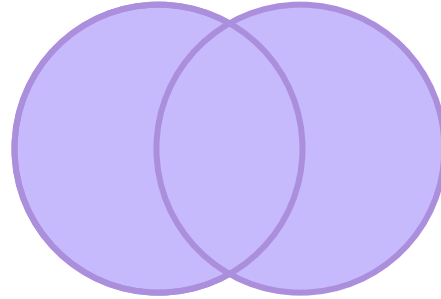


Joins

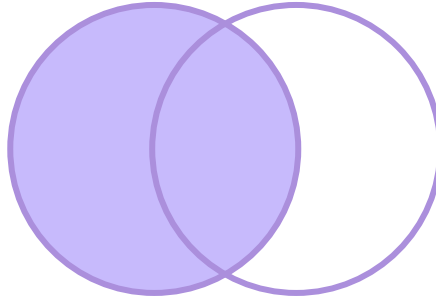
INNER



FULL OUTER



LEFT



CROSS JOIN

objects

item	price
sweater	25.00
yo-yo	6.00

people

name
Mary
Lucy
Bob

name	item	price
Mary	sweater	25.00
Mary	yo-yo	6.00
Lucy	sweater	25.00
Lucy	yo-yo	6.00
Bob	sweater	25.00
Bob	yo-yo	6.00

```
SELECT *  
FROM objects o  
      CROSS JOIN people p;
```

Self Join

pen_pals

name	country
Joyce	USA
Beatrice	Sweden
Mevis	Zimbabwe

```
SELECT a.name AS person1,  
       b.name AS person2
```

```
FROM pen_pals a  
     CROSS JOIN pen_pals b;
```

person1 person2

Joyce	Joyce
Joyce	Beatrice
Joyce	Mevis
Beatrice	Joyce
Beatrice	Beatrice
Beatrice	Mevis
Mevis	Joyce
Mevis	Beatrice
Mevis	Mevis

Self Join

```
SELECT a.name AS person1,  
       b.name AS person2  
  
FROM pen_pals a  
     CROSS JOIN pen_pals b
```

person1	person2
Joyce	Joyce
Joyce	Beatrice
Joyce	Mevis
Beatrice	Joyce
Beatrice	Beatrice
Beatrice	Mevis
Mevis	Joyce
Mevis	Beatrice
Mevis	Mevis

Self Join

```
SELECT a.name AS person1,  
       b.name AS person2  
  
FROM pen_pals a  
     CROSS JOIN pen_pals b  
  
WHERE person1 <> person2;
```

person1	person2
---------	---------

Joyce	Joyce
-------	-------

Joyce	Beatrice
-------	----------

Joyce	Mevis
-------	-------

Beatrice	Joyce
----------	-------

Beatrice	Beatrice
----------	----------

Beatrice	Mevis
----------	-------

Mevis	Joyce
-------	-------

Mevis	Beatrice
-------	----------

Mevis	Mevis
-------	-------

Self Join

```
SELECT a.name AS person1,  
       b.name AS person2  
  
FROM pen_pals a  
     CROSS JOIN pen_pals b  
  
WHERE person1 <> person2;
```

person1	person2
Joyce	Beatrice
Joyce	Mevis
Beatrice	Joyce
Beatrice	Mevis
Mevis	Joyce
Mevis	Beatrice

Self Join

```
SELECT a.name AS person1,  
       b.name AS person2  
  
FROM pen_pals a  
     CROSS JOIN pen_pals b  
  
WHERE person1 < person2;
```

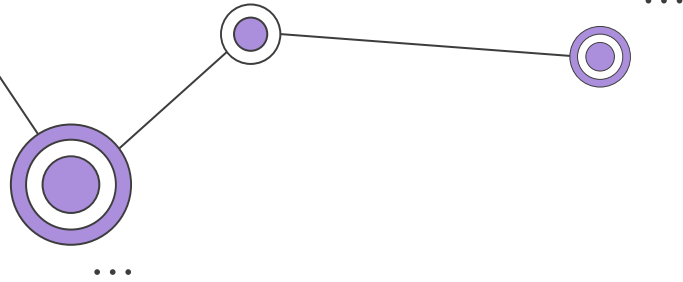


person1	person2
Joyce	Beatrice
Joyce	Mevis
Beatrice	Joyce
Beatrice	Mevis
Mevis	Joyce
Mevis	Beatrice

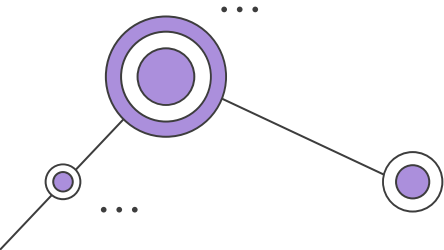
Self Join

```
SELECT a.name AS person1,  
       b.name AS person2  
  
FROM pen_pals a  
     CROSS JOIN pen_pals b  
  
WHERE person1 < person2;
```

person1	person2
Joyce	Mevis
Beatrice	Joyce
Beatrice	Mevis



Coding Demonstration



What does UNION do?

college

student	score
Javier	85
Pamela	91
Bin	79



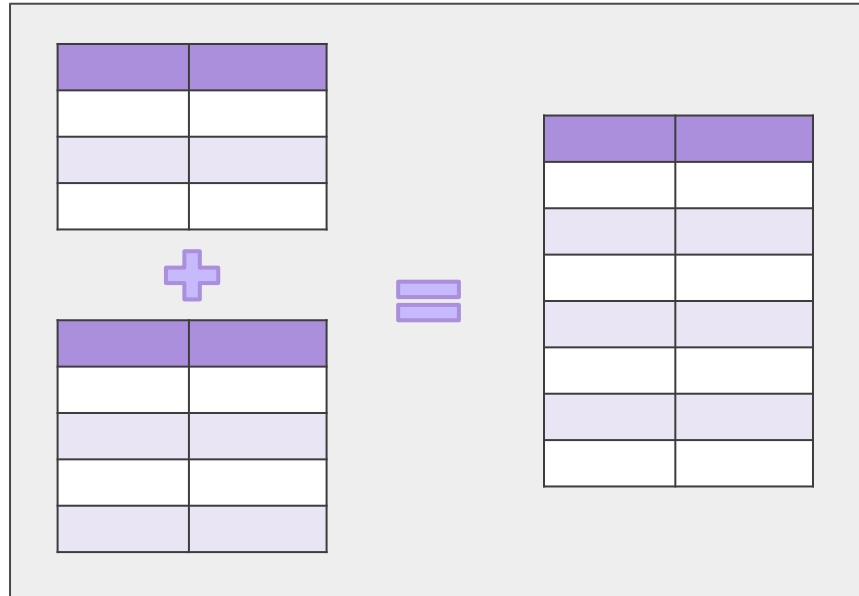
university

student	major	grade
Milly	Chemistry	95
Elena	Literature	68
Walter	Theater	90
Scout	Math	84

What does UNION do?

student	result
Javier	85
Pamela	91
Bin	79
Milly	95
Elena	68
Walter	84
Scout	84

The **UNION operator** combines information by stacking query results, vertically.



UNION in Action

college

student	score
Javier	85
Pamela	91
Bin	79



university

student	major	grade
Milly	Chemistry	95
Elena	Literature	68
Walter	Theater	90
Scout	Math	84

UNION in Action

First query

```
SELECT name,  
       score AS result  
FROM college
```

student	result
Javier	85
Pamela	91
Bin	79

UNION in Action

```
SELECT name,  
       score AS result  
FROM college
```

```
UNION ALL
```

UNION ALL
Operator

student	result
Javier	85
Pamela	91
Bin	79

UNION in Action

```
SELECT name,  
       score AS result  
FROM college
```

UNION ALL

```
SELECT name,  
       grade AS result  
FROM university;
```

student	result
Javier	85
Pamela	91
Bin	79

UNION in Action

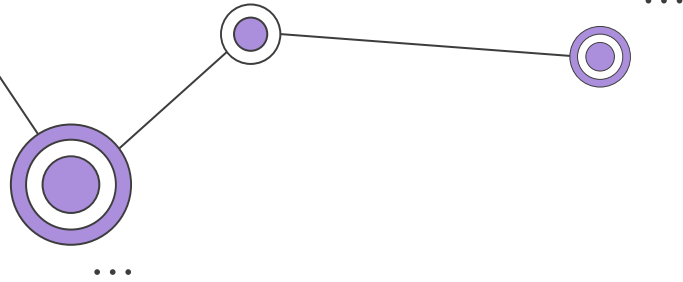
```
SELECT name,  
        score AS result  
FROM college
```

UNION ALL

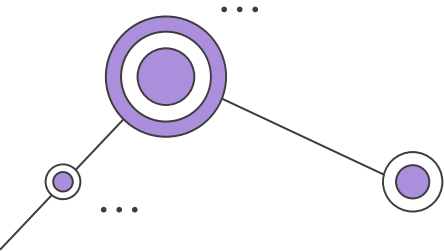
```
SELECT name,  
        grade AS result  
FROM university;
```

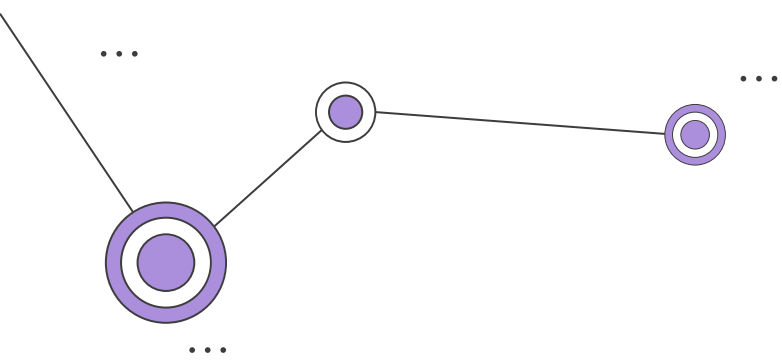
Second query

student	result
Javier	85
Pamela	91
Bin	79
Milly	95
Elena	68
Walter	84
Scout	84

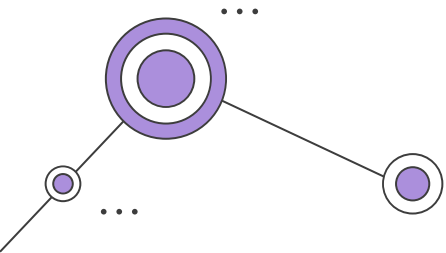


Coding Demonstration





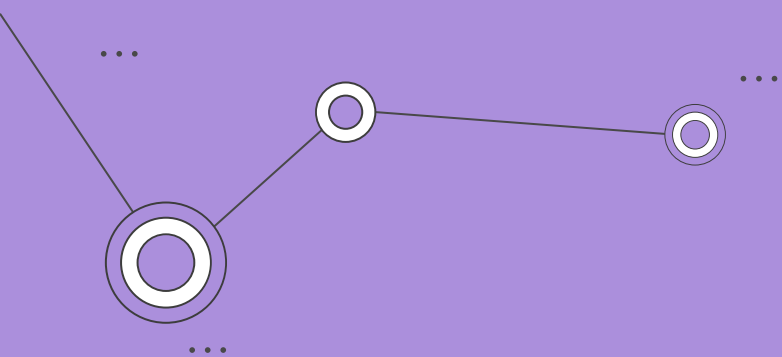
5-Minute Break



03

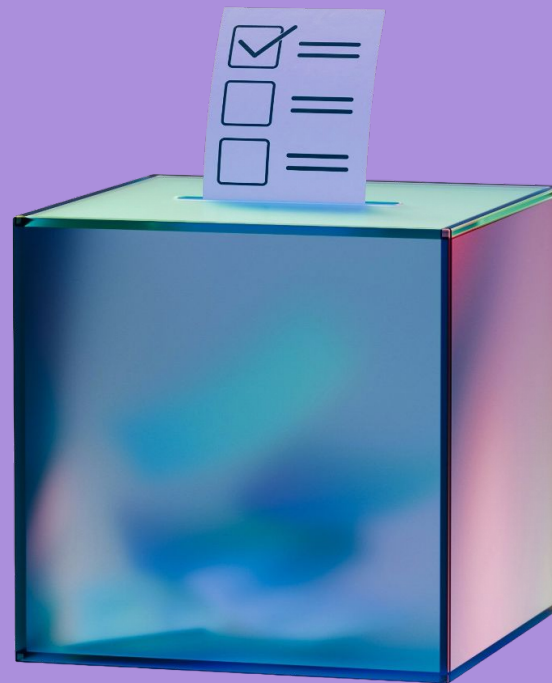
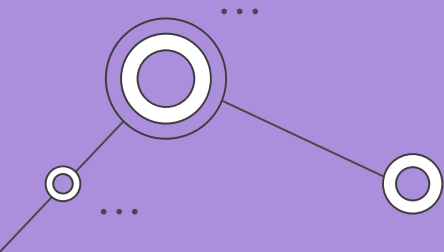
Window Functions





Poll #3

What do window functions do?



Determining Popularity

election_results

name votes	
John	55
Vickie	102
David	89
Elaine	55
Chris	74
Rob	31
Stacy	116

```
SELECT *  
  
FROM election_results  
  
ORDER BY votes DESC;
```


Determining Popularity

election_results

name votes	
John	55
Vickie	102
David	89
Elaine	55
Chris	74
Rob	31
Stacy	116

```
SELECT  
    *,
```

```
FROM election_results;
```

Determining Popularity

election_results

name	votes
John	55
Vickie	102
David	89
Elaine	55
Chris	74
Rob	31
Stacy	116

```
SELECT
```

```
*,
```

```
RANK()
```

Provides rankings;
keeps ties

```
FROM election_results;
```

Determining Popularity

election_results

name	votes
John	55
Vickie	102
David	89
Elaine	55
Chris	74
Rob	31
Stacy	116

```
SELECT
    *,
    RANK() OVER (

    ) AS popularity
FROM election_results;
```

OVER keyword
Alerts SQL of window function

Determining Popularity

`election_results`

name	votes
John	55
Vickie	102
David	89
Elaine	55
Chris	74
Rob	31
Stacy	116

```
SELECT
    *,
    RANK() OVER (
        ORDER BY votes DESC
    ) AS popularity
FROM election_results;
```

How we want to **sort** the data
before performing the window
calculations

Determining Popularity

`election_results`

name	votes	popularity
Stacy	116	1
Vickie	102	2
David	89	3
Chris	74	4
John	55	5
Elaine	55	5
Rob	31	7

`RANK ()` keeps ties

Determining Popularity

election_results

name	votes
John	55
Vickie	102
David	89
Elaine	55
Chris	74
Rob	31
Stacy	116

Determining Popularity

election_results

name	role	votes
John	Treasurer	55
Vickie	President	102
David	Secretary	89
Elaine	Secretary	55
Chris	President	74
Rob	President	31
Stacy	Treasurer	116

```
SELECT
    *,
    RANK() OVER (
        ORDER BY votes DESC
    ) AS popularity
FROM election_results;
```

Determining Popularity

election_results

name	role	votes
John	Treasurer	55
Vickie	President	102
David	Secretary	89
Elaine	Secretary	55
Chris	President	74
Rob	President	31
Stacy	Treasurer	116

```
SELECT
    *,
    RANK() OVER (

        ORDER BY votes DESC
    ) AS popularity
FROM election_results;
```


Determining Popularity

election_results

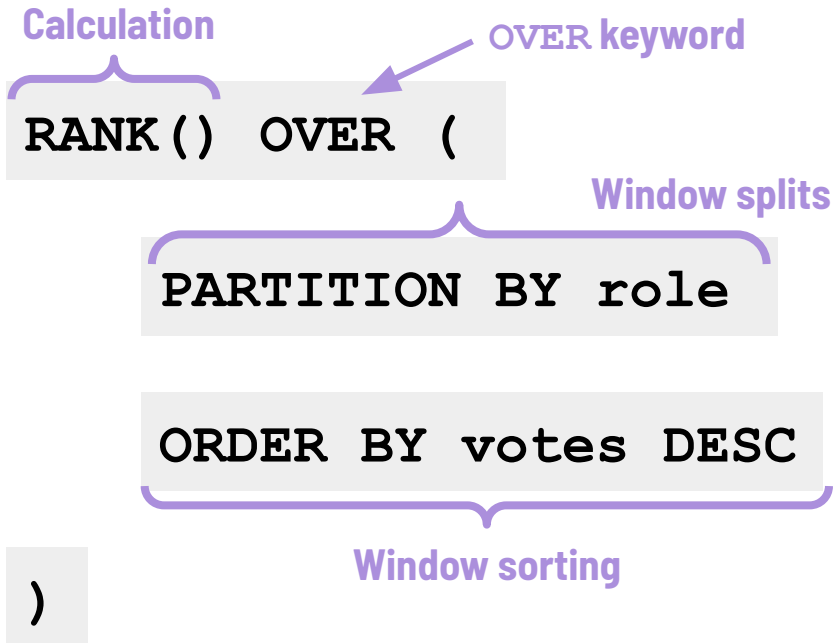
name	role	votes
John	Treasurer	55
Vickie	President	102
David	Secretary	89
Elaine	Secretary	55
Chris	President	74
Rob	President	31
Stacy	Treasurer	116

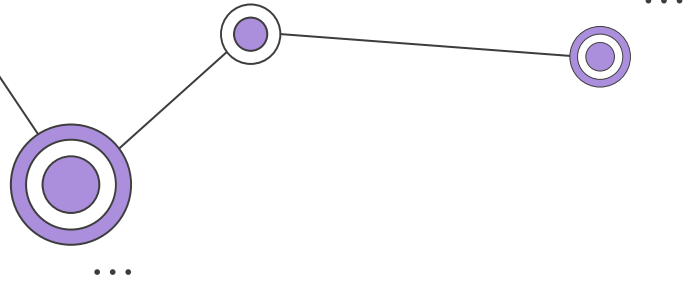
```
SELECT
  *,
  RANK() OVER (
    PARTITION BY role
    ORDER BY votes DESC
  ) AS popularity
FROM election_results;
```

How we want to **split** the data
into windows

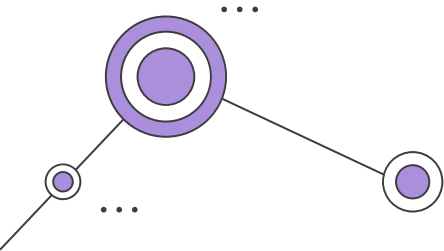
Window Functions

Window functions do calculations across rows, returning one value for each row.





Coding Demonstration



Group Summaries

election_results

name	role	votes
John	Treasurer	55
Vickie	President	102
David	Secretary	89
Elaine	Secretary	55
Chris	President	74
Rob	President	31
Stacy	Treasurer	116

```
SELECT  
*,
```

```
FROM election_results;
```

Group Summaries

election_results

name	role	votes
John	Treasurer	55
Vickie	President	102
David	Secretary	89
Elaine	Secretary	55
Chris	President	74
Rob	President	31
Stacy	Treasurer	116

```
SELECT
    *,
    AVG(votes) OVER (
        ) AS avg_votes
FROM election_results;
```

Group Summaries

election_results

name	role	votes
John	Treasurer	55
Vickie	President	102
David	Secretary	89
Elaine	Secretary	55
Chris	President	74
Rob	President	31
Stacy	Treasurer	116

```
SELECT
    *,
    AVG(votes) OVER (
        PARTITION BY role
    ) AS avg_votes
FROM election_results;
```

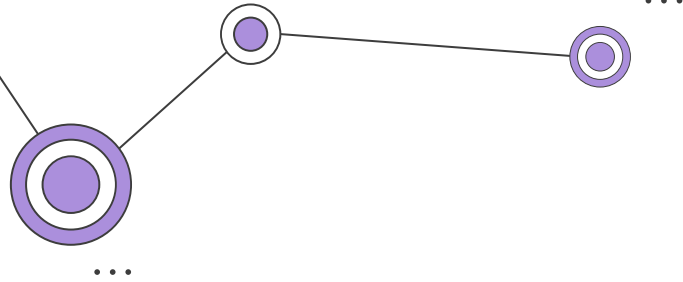
Group Summaries

election_results

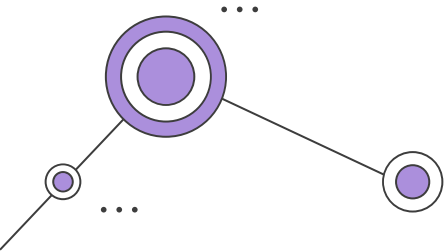
name	role	votes	avg_votes
John	Treasurer	55	85.5
Vickie	President	102	69.0
David	Secretary	89	72.0
Elaine	Secretary	55	72.0
Chris	President	74	69.0
Rob	President	31	69.0
Stacy	Treasurer	116	85.5

```
SELECT
    *,
    AVG(votes) OVER (
        PARTITION BY role
    ) AS avg_votes
FROM election_results;
```

Window functions give us
one value per row.



Coding Demonstration



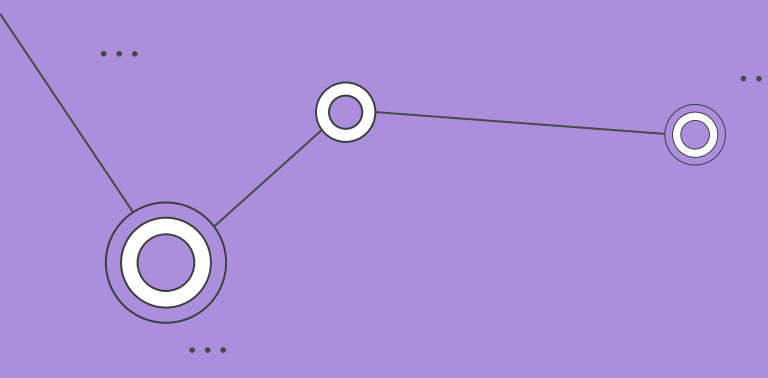
5-Minute Break



04

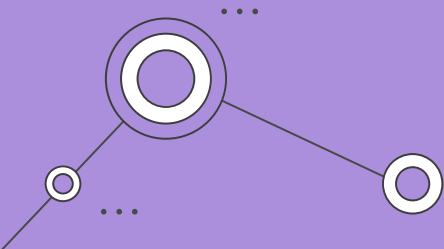
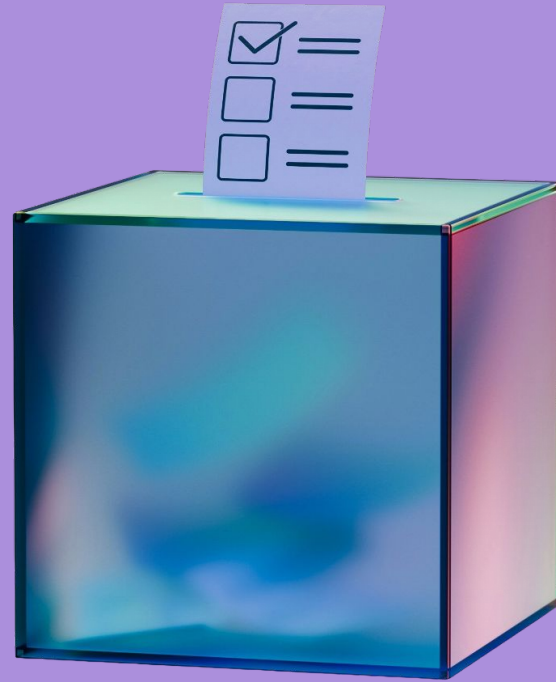
Analysis Patterns





Poll #4

Which of these tasks have you performed in your work?



Analysis Patterns

01

Top-N per Group

...

02

Outlier Detection

...

03

Cohort Retention

...

04

MoM Growth

...



Analysis Patterns

events

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Top-N per Group

Who are the top 2 users per category by total value?

1. Aggregate user totals across categories.
2. Rank the total aggregates.
3. Filter down to the top 2 users per category.

Top-N per Group

1. **Aggregate** user totals across categories.

```
WITH totals AS (  
    SELECT  
        user_id,  
        category,  
        SUM(value) AS total_value  
    FROM events  
    GROUP BY category, user_id  
)
```

First CTE

Top-N per Group

2. **Rank** the total aggregates.

```
, ranked AS (  
    SELECT  
        user_id,  
        category,  
        total_value  
        RANK() OVER (  
            PARTITION BY category  
            ORDER BY total_value  
        ) AS cat_rank  
    FROM totals  
)
```

Second CTE

Top-N per Group

3. **Filter** down to the top 2 users per category.

Main query

```
SELECT
    category,
    user_id,
    total_value
FROM ranked
WHERE cat_rank <= 2
ORDER BY category, cat_rank;
```

Top-N per Group

events

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

category	user_id	value
A	3	50
A	1	25
B	2	27
C	3	5

Top-N per Group

events

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

category	user_id	value
A	3	50
A	1	25
B	2	27
C	3	5

Top-N per Group

AGGREGATE

RANK

FILTER

Outlier Detection

events

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Outlier Detection

events

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Outlier Detection

Which users are behaving unusually compared to others in the same category?

1. Compute necessary summary statistics by category.
2. Compare individual values to the aggregate.
3. Flag outliers.

Choose desired definition.
Here, **more than 2 x average.**

Outlier Detection

1. Compute necessary **summary statistics** by category.

```
WITH stats AS (  
    SELECT  
        category,  
        AVG(value) AS avg_value  
    FROM events  
    GROUP BY category  
)
```

CTE



Outlier Detection

2. **Compare** individual values to the aggregate.

```
SELECT
  e.*,
  CASE
    WHEN value > 2 * s.avg_value
```

Outlier Detection

3. **Flag** outliers.

```
SELECT
    e.*,
    CASE
        WHEN value > 2 * s.avg_value
            THEN 'outlier'
        ELSE 'normal'
    END AS flag
FROM events e
LEFT JOIN stats s
    ON e.category = s.category;
```

Outlier Detection

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Outlier Detection

user_id	event_date	category	value	flag
1	2026-01-03	A	10	normal
1	2026-02-10	A	15	normal
2	2026-01-04	A	20	normal
2	2026-01-05	B	7	normal
2	2026-02-20	B	20	normal
3	2026-01-07	A	50	outlier
3	2026-03-01	C	5	normal

Outlier Detection

user_id	event_date	category	value	flag
1	2026-01-03	A	10	normal
1	2026-02-10	A	15	normal
2	2026-01-04	A	20	normal
2	2026-01-05	B	7	normal
2	2026-02-20	B	20	normal
3	2026-01-07	A	50	outlier
3	2026-03-01	C	5	normal

Outlier Detection

SUMMARIZE

COMPARE

FLAG

Cohort Retention

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Cohort Retention

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Cohort Retention

1. **Cohort:** Who started at the same time?

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Cohort Retention

1. **Cohort:** Who started at the same time?

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

Cohort Retention

1. **Cohort:** Who started at the same time?

user_id	event_date	category	value	cohort
1	2026-01-03	A	10	January
1	2026-02-10	A	15	January
2	2026-01-04	A	20	January
2	2026-01-05	B	7	January
2	2026-02-20	B	20	January
3	2026-01-07	A	50	January
3	2026-03-01	C	5	January

Cohort Retention

2. **Time alignment:** How far along is each user relative to their start?

user_id	event_date	category	value	cohort
1	2026-01-03	A	10	January
1	2026-02-10	A	15	January
2	2026-01-04	A	20	January
2	2026-01-05	B	7	January
2	2026-02-20	B	20	January
3	2026-01-07	A	50	January
3	2026-03-01	C	5	January

Cohort Retention

2. **Time alignment:** How far along is each user relative to their start?

user_id	event_date	category	value	cohort	month
1	2026-01-03	A	10	January	0
1	2026-02-10	A	15	January	1
2	2026-01-04	A	20	January	0
2	2026-01-05	B	7	January	0
2	2026-02-20	B	20	January	1
3	2026-01-07	A	50	January	0
3	2026-03-01	C	5	January	2

Cohort Retention

3. **Count:** How many users are active at each time point for each cohort?

cohort	month	tally
January	0	3
January	1	2
January	2	1

Cohort Retention

4. **Retention:** What fraction of the original cohort is still active?

cohort	month	tally
January	0	3
January	1	2
January	2	1

Cohort Retention

4. **Retention:** What fraction of the original cohort is still active?

cohort	month	tally	retention
January	0	3	1.00
January	1	2	0.67
January	2	1	0.33

Cohort Retention

COHORT

TIME ALIGNMENT

COUNT

RETENTION

MoM Growth

How is the total value changing **month over month** (MoM)?

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

MoM Growth

1. Build **monthly totals**.

user_id	event_date	category	value
1	2026-01-03	A	10
1	2026-02-10	A	15
2	2026-01-04	A	20
2	2026-01-05	B	7
2	2026-02-20	B	20
3	2026-01-07	A	50
3	2026-03-01	C	5

MoM Growth

1. Build **monthly totals**.

year	month	total_value
2024	1	87
2024	2	35
2024	3	5

MoM Growth

2. Line up each month with the **previous month**.

year	month	total_value
2024	1	87
2024	2	35
2024	3	5

MoM Growth

2. Line up each month with the **previous month**.

year	month	total_value	prev_month
2024	1	87	NULL
2024	2	35	87
2024	3	5	35

MoM Growth

3. Calculate the **growth**.

year	month	total_value	prev_month
2024	1	87	NULL
2024	2	35	87
2024	3	5	35

MoM Growth

3. Calculate the **growth**.

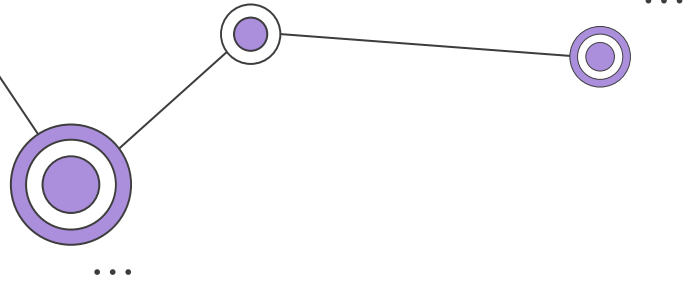
year	month	total_value	prev_month	mom_change
2024	1	87	NULL	NULL
2024	2	35	87	-52
2024	3	5	35	-30

MoM Growth

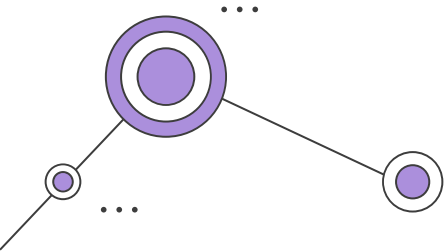
MONTHLY TOTALS

PREVIOUS MONTH

GROWTH



Coding Demonstration



Recap: Learning Objectives



Clean and transform data
directly in SQL

- Find and filter **missing values**. `IS NULL`
- Convert between **data types**. `CAST`
- Manipulate **strings**. `REPLACE TRIM LOWER`
- Work with **dates**. `EXTRACT INTERVAL DATE_FORMAT`
- **Bucket data** into categories. `CASE WHEN... THEN... END`

Recap: Learning Objectives



Combine and
query from
multiple tables

- Write **subqueries** and **derived tables**. `SELECT... (SELECT...)`
- Build common table expressions (**CTEs**). `WITH my_cte AS...`
- Do **cross joins** and **self joins**. `CROSS JOIN`
- Stack results with **UNION**. `UNION UNION ALL`

Recap: Learning Objectives



Perform complex
aggregations
and rankings

Build **window functions**... **OVER**...

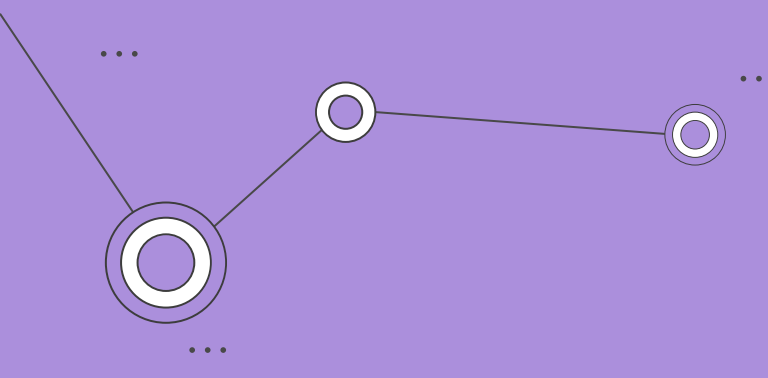
- For groups. **PARTITION BY**
- With sorting. **ORDER BY**
- To **rank** individual rows. **RANK ()** **LAG ()**
- To **summarize** or compute a **running tally**. **SUM ()** **AVG ()**

Recap: Learning Objectives



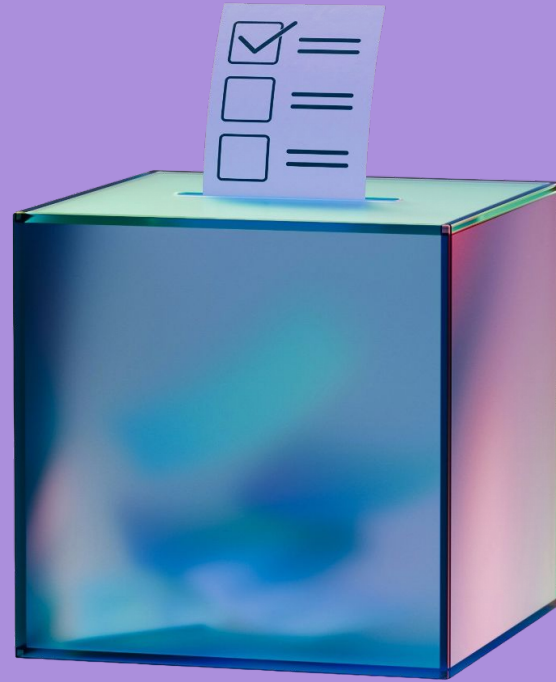
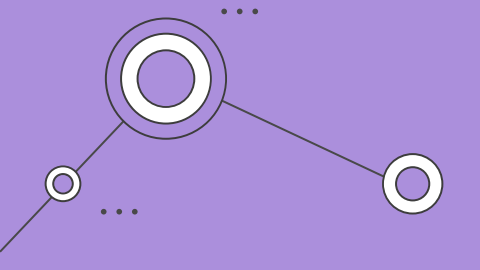
Solve **real-world data problems** through common analysis patterns

- Top-N per group **Aggregate → Rank → Filter**
- Outlier detection **Summarize → Compare → Flag**
- Cohort retention **Cohort → Time alignment → Count → Retention**
- MoM growth **Monthly totals → Previous month → Growth**



Poll #5

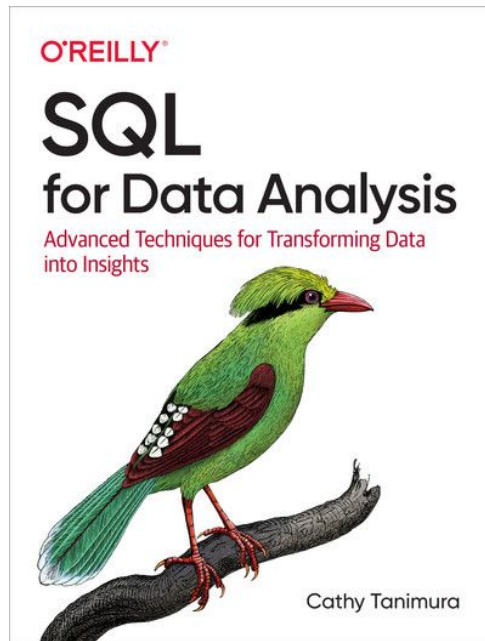
Which methods are you most looking forward to applying?



Continuing Your Journey

For best results, you should:

- **Practice** these SQL commands.
- **Apply** each skill to your work.
- Explore other SQL **resources**.
 - ***SQL for Data Analysis*** book, Cathy Tanimura.



Continuing Your Journey

For best results, you should:

- **Practice** these SQL commands.
- **Apply** each skill to your work.
- Explore other SQL **resources**.
 - ***SQL for Data Analysis*** book, Cathy Tanimura.
 - ***Advanced SQL Queries in 90 Minutes***, Alice Zhao



Stay in Touch



Kimberly Fessel



kimfetti



Kimberly Fessel



www.drkimdata.com

CREDITS: This presentation template was created by [Slidesgo](#), including icons by [Flaticon](#), infographics & images by [Freepik](#) and illustrations by [Stories](#)